# Web Scraping Tutorial : rvest, RSelenium & Rcrawler

Cathelyn Tan

## Table of contents

**1 Introduction : What is Web Scraping? 2**
1.1 Prerequisites . . . 2

**2 Ethical and legal considerations 2**

**3 Understanding HTML Basics 2**
3.1 HTML Elements . . . 2
3.2 HTML Attributes . . . 2

**4 CSS 3**
4.1 Classes . . . 3
4.2 IDs . . . 3

**5 Web Scraping in R 3**
5.1 Extracting data . . . 3
5.1.1 CSS Selectors . . . 4
5.2 Extracting Nested Elements . . . 5
5.3 Text and Attributes . . . 5
5.3.1 `html_text2()` . . . 5
5.3.2 `html_attr()` . . . 5
5.4 Scraping tables . . . 6

**6 Advanced Scraping with `RSelenium` 6**
6.1 Prerequisites . . . 6
6.2 Scraping static pages . . . 6
6.2.1 Accessing Elements . . . 6
6.2.2 Extracting Data from Elements . . . 7
6.3 Scraping dynamic pages . . . 7

**7 Using `Rcrawler` 7**
7.1 Scraping data with XPath . . . 7
7.2 Scraping data with CSS selectors . . . 7
7.3 Scraping data with XPath . . . 8
7.4 Handling scraped data . . . 8

**8 Conclusion : when to use rvest, RSelenium or Rcrawler? 8**

**9 Citations 8**

---

# 1 Introduction : What is Web Scraping?

Web scraping is a very useful tool used for extracting data from websites. Often, data providers publish their data in a convenient readable formatted tables. However, not all of these sites provide means to download and collect the data, that is when web scraping comes in handy. We will be learning basics of HTML and CSS along with the use of R packages such as `rvest`, `RSelenium` and `Rcrawler` for web scraping.

## 1.1 Prerequisites

Before we begin, install the required R packages:

```
install.packages("tidyverse")
install.packages("rvest")
```

Load the libraries:

```
library(tidyverse)
library(rvest)
```

# 2 Ethical and legal considerations

There are limits to which web scraping tools can be used, that is only if the data is **public**, **non-personal**, and **factual**. These factors are critical as they are connected to the **terms and conditions** of the website, **copyright** and **personally identifiable information**. If the data does not fill the requirements described previously, and the use of scraping the data is monetary, then consulting with a lawyer is necessary. In any case, we should respect the resources of the server hosting the site being scraped.

# 3 Understanding HTML Basics

HTML stands for HyperText Markup Language, used to describe the structure and content of web pages. HTML uses tags surrounded by < >, an example of HTML code can look like this :

```
<html>
    <title>WEB SCRAPING TUTORIAL</title>
<body>
  <h1 id='first'>this is a heading</h1>
  <p><b>we can bold text like this</b></p>
</body>
```

## 3.1 HTML Elements

HTML elements consist of:

- start tag : <tag>
- end tag : </tag>
- content : Text or other elements between the start and end tags.

Tags can be **block** or **inline**, block tags (forms structure of the content) are <h1>, <section>, <p> and <ol>. Inline tags (format text inside block tags) are <b>, <i>, <a>.

## 3.2 HTML Attributes

Attributes define extra information about elements. They are written as name='value', some of important attributes are **id**, **class**, **href** and **src**. These are essential to identify specific elements when scraping.

# 4 CSS

CSS is short for Cascading Style Sheet. While HTML provides the content and structure of a web page, CSS is used to style HTML elements, controlling aspects like color, positioning, and layout. Two concepts we need to learn before we delve into the R web scraping code are **classes** and **ids**.

## 4.1 Classes

CSS classes group similar elements together for consistent styling. For example, if we want several paragraphs to appear in color blue, we could assign them a common class like blue-text and define the style once in a separate CSS file :

```
.blue-text {
  color: blue;
}
```

## 4.2 IDs

CSS ids are used to uniquely identify a single element. For example :

```
<p id="special" >This is a special tag.</p>
```

this makes it easier to target specific elements when scraping.

# 5 Web Scraping in R

Make sure all the necessary packages (shown in the introduction part) has been installed and loaded into the R environment.

## 5.1 Extracting data

The first step is to read the HTML from a specified URL using `read_html()` function, which takes the URL of the website you want to scrape as an argument and returns an `xml_document` object.

```
html <- read_html("https://cathelyn06.github.io/webscrapingRcath/webscrapingresources.html")
html
```

```
{html_document}
<html lang="en">
[1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8 ...
[2] <body>\n    <p>Everyone can learn about web scraping.</p>\n  </body>
```

Above is an example `xml_document`, this structure contains the tags and contents of the web page, which we will analyze to extract the data.

You can also write HTML inline using `minimal_html()` like this :

```
example <- minimal_html("
  <p>Hello!</p>
  <ol>
    <li>This is an ordered list</li>
  </ol>
")
example
```

```
{html_document}
<html>
[1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8 ...
[2] <body>\n<p>Hello!</p>\n  <ol>\n<li>This is an ordered list</li>\n  </ol>\ ...
```

### 5.1.1 CSS Selectors

CSS selectors are key to locating specific elements in the HTML. The rvest package uses CSS selectors to target HTML elements. Here are examples of some basic selectors:

- p selects all <p> elements.
- .class-name selects all elements with the class class-name (ex : .important)
- #id selects the element with the id attribute which is unique, so only select that single unique element (ex : #first).

```
example2 <- read_html("https://cathelyn06.github.io/webscrapingRcath/cssselectorsexample.html")
example2 %>% html_elements("p")
```

```
{xml_nodeset (3)}
[1] <p id="first">This is a paragraph with an ID.</p>
[2] <p class="important">This is an important paragraph with a class.</p>
[3] <p class="important">This is another important paragraph with class.</p>
```

```
example2 %>% html_elements(".important")
```

```
{xml_nodeset (2)}
[1] <p class="important">This is an important paragraph with a class.</p>
[2] <p class="important">This is another important paragraph with class.</p>
```

```
example2 %>% html_elements("#first")
```

```
{xml_nodeset (1)}
[1] <p id="first">This is a paragraph with an ID.</p>
```

we have seen the use of `html_elements()` (which is also called html_nodes() in older version), another important function is `html_element()`.

- **html_elements()**: Returns all matching elements as a vector. If no match is found, it returns a vector of length 0.
- **html_element()**: Returns only the first matching element. If no match is found, it returns a NA (missing value).

```
example2 %>% html_elements(".important") %>% html_text2()
```

```
[1] "This is an important paragraph with a class."
[2] "This is another important paragraph with class."
```

```
example2 %>% html_element(".important") %>% html_text2()
```

```
[1] "This is an important paragraph with a class."
```

## 5.2 Extracting Nested Elements

If you want to extract data which is **nested** within other elements, both `html_elements()` and `html_element()` may be used. For instance, you want to get the grades of students from a list. Here's an example where each list item contains students' name and grades :

```
nested_elements <- minimal_html("
  <ul>
    <li><b>Alex</b> has a score of <span class='grade'>97%</span></li>
    <li><b>Cathelyn</b> has a score of </li>
    <li><b>Jane</b> has a score of <span class='grade'>88.5%</span></li>
  </ul>
")
```

To extract the name of students and their grades, we can first make a vector where each element corresponds to a different character :

```
characters <- nested_elements %>% html_elements("li")
characters %>% html_element("b") #gives the students' name
```

```
{xml_nodeset (3)}
[1] <b>Alex</b>
[2] <b>Cathelyn</b>
[3] <b>Jane</b>
```

```
characters %>% html_element(".grade") #gives their grades
```

```
{xml_nodeset (3)}
[1] <span class="grade">97%</span>
[2] NA
[3] <span class="grade">88.5%</span>
```

## 5.3 Text and Attributes

### 5.3.1 `html_text2()`

This function extracts plain text contents of an HTML element :

```
html%>%
  html_element("p") %>%
  html_text2()
```

```
[1] "Everyone can learn about web scraping."
```

### 5.3.2 `html_attr()`

This function extracts data from attributes and always returns a **string**. For example :

```
example2%>%
  html_elements("p.important") %>%
  html_attr("class")
```

```
[1] "important" "important"
```

## 5.4 Scraping tables

There are times where data is stored in HTML tables, and `rvest` provides a handy function to extract table data by using `html_table()`, which returns it as a tibble. Here is an example :

```
example_table <- minimal_html("
  <table class='mytable'>
    <tr><th>name</th>   <th>midterm grade</th></tr>
    <tr><td>Alex</td> <td>97</td></tr>
    <tr><td>Cathelyn</td> <td>92</td></tr>
  </table> ")
example_table %>% html_element("table") %>% html_table()
```

```
# A tibble: 2 x 2
  name     `midterm grade`
  <chr>              <int>
1 Alex                  97
2 Cathelyn              92
```

# 6 Advanced Scraping with `RSelenium`

RSelenium is another powerful tool in R for web scraping, especially when dealing with dynamic web pages which requires user interactions or JavaScript related. Unlike rvest, RSelenium allows you to control a web browser programmatically, enabling scraping of content that loads dynamically such as clicking buttons or scrolling.

## 6.1 Prerequisites

1. Install Java
2. Download the ChromeDriver and Selenium server
3. Install RSelenium in R

```
install.packages('RSelenium')
```

## 6.2 Scraping static pages

Then load RSelenium and connect the to Selenium server in R and navigate to a web page.

```
library(RSelenium)
rem_driver <- remoteDriver(remoteServerAddr = 'localhost', #connect to selenium server
                           browserName = 'chrome',
                           port=4444L)
rem_driver$open()
rem_driver$navigate("https://www.example.com") #navigate to a web page
```

### 6.2.1 Accessing Elements

RSelenium offers several ways to identify and access elements, such as by ID, class or **CSS selectors** :

1. By **ID** :

```
element_id <- rem_driver$findElement(using = 'id', value = 'element_id_here')
```

2. By **Class** :

```
element_class <- rem_driver$findElement(using = 'class', value = 'class_name_here')
```

3. By **CSS Selector** :

```
 element_css <- rem_driver$findElements(using = 'css', value = 'css_selector_here')
```

#### 6.2.2 Extracting Data from Elements

Once an element is selected, you can extract its data. For example:

**Extract Attribute (e.g., image link)**: getElementAttribute() function can be used to get image link.

```
attribute_value <- elements_css[[1]]$getElementAttribute('attribute_name_here')
```

**Extract Inner Text**: To extract the text inside an element, use getElementText():

```
inner_text <- element_class[[1]]$getElementText()
```

### 6.3 Scraping dynamic pages

Dynamic pages often require user interaction, such as clicking or scrolling, to load content. With RSelenium, you can automate these interactions.

**Clicking Elements** :

```
rem_driver$findElements(using = 'css', value = 'css_selector_here')[[1]]$clickElement()
```

**Submitting forms or entering text** :

```
rem_driver$findElement(using = 'css', value = 'input_selector_here')$sendKeysToElement(list(
  "text_here", key = "enter"))
```

**Scrolling** :

```
rem_driver$executeScript("window.scrollTo(0, document.body.scrollHeight);")
```

## 7 Using `Rcrawler`

`Rcrawler` is another R package that helps us harvest information from the web. `Rcrawler`is used to scrape network graph related tasks.

### 7.1 Scraping data with XPath

```
Rcrawler(Website = "https://your-website-url.com", # Replace with the actual URL
         no_cores = 4,
         no_conn = 4,
         ExtractXpathPat = c("//h2[@class='your-title-class']", "//div[@class='your-content
                              -class']"),
         # Replace with XPath that targets the title and content element on your website.

         PatternsNames = c("Title", "Content"))
```

### 7.2 Scraping data with CSS selectors

```
Rcrawler(Website = "https://your-website-url.com", # Replace with the actual URL
         no_cores = 4,
         no_conn = 4,
         ExtractCSSPat = c(".your-title-class", ".your-content-class"),
         # Replace with the CSS class that targets the title and content element on your website.
         PatternsNames = c("Title", "Content"))
```

### 7.3 Scraping data with XPath

```
links <- ContentScraper(Url = "https://your-website-url.com/page1", # Replace with actual URL
                        XpathPatterns = "//*/a/@href",
                        ManyPerPattern = TRUE)

# Scraping Links from the whole website
Rcrawler(Website = "https://your-website-url.com/",
         ExtractXpathPat = "//*/a/@href",
         # Replace with the XPath pattern that targets the links (URLs) on your website.
         ManyPerPattern = TRUE)
```

### 7.4 Handling scraped data

**Extracting Titles** :

```
Title <- unlist(lapply(DATA, `[[`, 1))
```

**Extracting Content** :

```
Content <- unlist(lapply(DATA, `[[`, 2))
```

**Converting Data to a Data frame** :

```
df <- data.frame(do.call("rbind", DATA))
```

## 8 Conclusion : when to use rvest, RSelenium or Rcrawler?

| Tool | Best for | When to use | Example |
|---|---|---|---|
| `rvest` | Static websites | • Data can be found directly in HTML source.<br>• No JavaScript interaction required. | scraping **product prices** from an e-commerce site. |
| `RSelenium` | Dynamic websites | • Scrolling, others.<br>• Requires JavaScript handling | Extract job listings from site with "**Load More**" button. |
| `Rcrawler` | Large-scale data scraping | • Scraping multiple pages or entire webpage.<br>• Extracting large datasets. | Scraping **news articles** from a blog |

## 9 Citations


Hadley Wickham, M. Ç.-R. (n.d.). *24 web scraping.* 24 Web scraping – R for Data Science (2e). https://r4ds.hadley.nz/webscraping.html#nesting-selections

Pascual, C. (2024, May 10). *Tutorial: Web scraping in R with rvest.* Dataquest. https://www.dataquest.io/blog/web-scraping-r-rvest/

Peters, R. (2023, November 27). *Getting started with rselenium.* ScrapingBee. https://www.scrapingbee.com/blog/getting-started-with-rselenium/

Khalil, S., & Fakir, M. (2017). RCrawler: An R package for parallel web crawling and scraping. SoftwareX, 6, 98-106.